

Claudecode 介绍讲稿

Zhiyu Lu

2026 年 5 月

本文目标：详细说明 Claude Code 与已有工具的区别与优势，解析其工作原理，并结合科研实践探讨 Skills 设计。

一、大语言模型的使用范式与演进

1.1 API 接口的基本认知

在介绍任何具体工具之前，有必要先建立一个认知基础：大语言模型的输入，究竟由哪些层次组成？

表面上看，你打一段话，模型回一段话。但实际上，一次 API 调用的上下文由五个层次叠加而成：

训练层（权重）	← 任何 prompt 都无法触达
核心价值 / 绝对限制 / 能力边界	
平台注入层（meta-system）	← 用户和开发者均不可见
Anthropic / OpenAI 的内部指令块	
operator system prompt	← 开发者可读可写
user message	← 用户可读可写
assistant message（历史轮次）	

这个分层结构揭示了网页端与 API 端的本质差异：

- 网页端（Claude.ai、ChatGPT 等）：用户只能写 user message。模型内置了搜索、PDF 阅读、代码执行等工具，但这些工具由官方预设，用户无法替换。
- API 端：开发者可以直接编写 system prompt，自主决定给模型什么背景、哪些工具、何种行为约束。代价是：原生工具消失，一切都要自己搭建。

Sider 这类侧边栏插件，本质上是对网页端的轻量包装——它读取你当前页面的内容，然后作为 user message 的上下文喂给模型。Cursor 和 Copilot 则更进一步，通过 API 将你的代码文件注入 system prompt，让模型能够直接感知你的工作区。

1.2 范式演进：从提示词到工具，再到智能体

第一阶段：Prompt Engineering（2021 年）

早期模型能力有限，完成复杂任务的核心手段是设计精巧的提示词，预先写好工作流的每一步。这种方法有明显的天花板：提示词越长越难维护，模型遵循复杂指令的能力也很快触及上限。

第二阶段：工作流与工具集成（2023—2024 年）

两条并行的线索推动了这个阶段的发展：

模型本身的能力跃迁：更长的上下文窗口（对应更低的幻觉率）、推理能力（思维链，让模型自行决定中间步骤）、以及工具调用能力（Function Calling，让模型能读懂工具的输入输出格式，主动决定调用哪个工具）。

工具生态的建立：

- 工作流类工具（如 Coze）：将复杂任务拆解为多个子任务，由代码硬连接各模型间的通信。优点是稳定可控，缺点是流程僵硬，任何非预设的情况都会失败。
- 辅助类工具（如 Copilot、Cursor）：可以读取用户当前的工作区文件，直接在编辑器中插入或替换内容，大幅降低了上下文拼接的成本。
- Agent 类工具（如 Manus）：为模型配置一套预制的工具环境，让模型可以自主规划较长时间跨度的工作流，执行代码，访问网络。

然而，这个阶段的工具有一个共同的根本缺陷：输入和输出完全由用户控制，模型与用户的本地工作环境之间存在割裂。

模型或许能写出论文大纲，但无法在生成文本后，自动将其排版为 .docx 文件，或者调用 LaTeX 编译器完成论文编译。模型只能生成内容，却无法触及内容之外的世界。

第三阶段：模型与环境的深度集成（2024 年至今）

这一阶段的核心洞察是：既然任务设计可以由 AI 完成，代码执行也可以由 AI 完成，那么只需搭建一个预制框架，AI 就可以自主决定读哪些文件、用哪些工具、设计怎样的工作流。

这就是所谓“AI Agent”的落地实现。AI 不再是被动响应的工具，而是初步具备自主执行力的智能体。Claude Code 正是这一逻辑的完整体现。

二、Claude Code 的架构解析

Claude Code 在前段时间发生了源码泄露，为理解这套系统的设计原理提供了难得的机会。以下分析基于公开讨论与泄露内容。

2.1 System Prompt 的分层设计

泄露的源码揭示，Claude Code 的 system prompt 在代码中由 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 分成两段——这一设计极其关键：

静态区（全局 KV 缓存）

包含所有用户共享的内容：工具 schema、行为准则、输出格式要求。这部分对所有组织的所有用户只需计算一次，是显著的成本优化。内部 A/B 测试结果显示，硬性数字比模糊指令效果更好——例如“步骤间文字不超过 25 词”，比“保持简洁”约减少 1.2% 的 token 消耗。

动态区（每次请求重新注入）

包含当前项目状态、用户配置、工具权限、记忆摘要等会随使用变化的内容。

2.2 对话历史的真实结构

Claude Code 的对话历史并非普通聊天记录。每一轮实际上由三种内容交错构成：

1. 你打的字（真实的 user turn）
2. 工具执行结果，例如 `FileReadTool` 读到的文件内容，以 `tool_result` 的形式注入对话——这就是为什么 Claude 能“看到”文件
3. Bash 命令的输出，同样以 `tool_result` 方式注入

当对话过长时，Claude Code 会启动 **Compaction**（上下文压缩）：派生出一个子 Claude，用专门的摘要 prompt 把整段历史压缩为结构化摘要，再注入回对话。

值得注意的是：这个压缩过程不区分“用户说的话”和“工具读到的文件内容”——两者被一视同仁地压缩进摘要。这意味着，如果恶意文件中包含伪装成指令的文本，在 Compaction 后，它会以“正常对话内容”的身份持久存活，形成一种潜在的注入攻击面。这是使用此类工具时需要了解的安全边界。

2.3 Agentic Loop：三阶段工作逻辑

当你给 Claude 一个任务时，它会经历一个持续循环的三阶段过程：

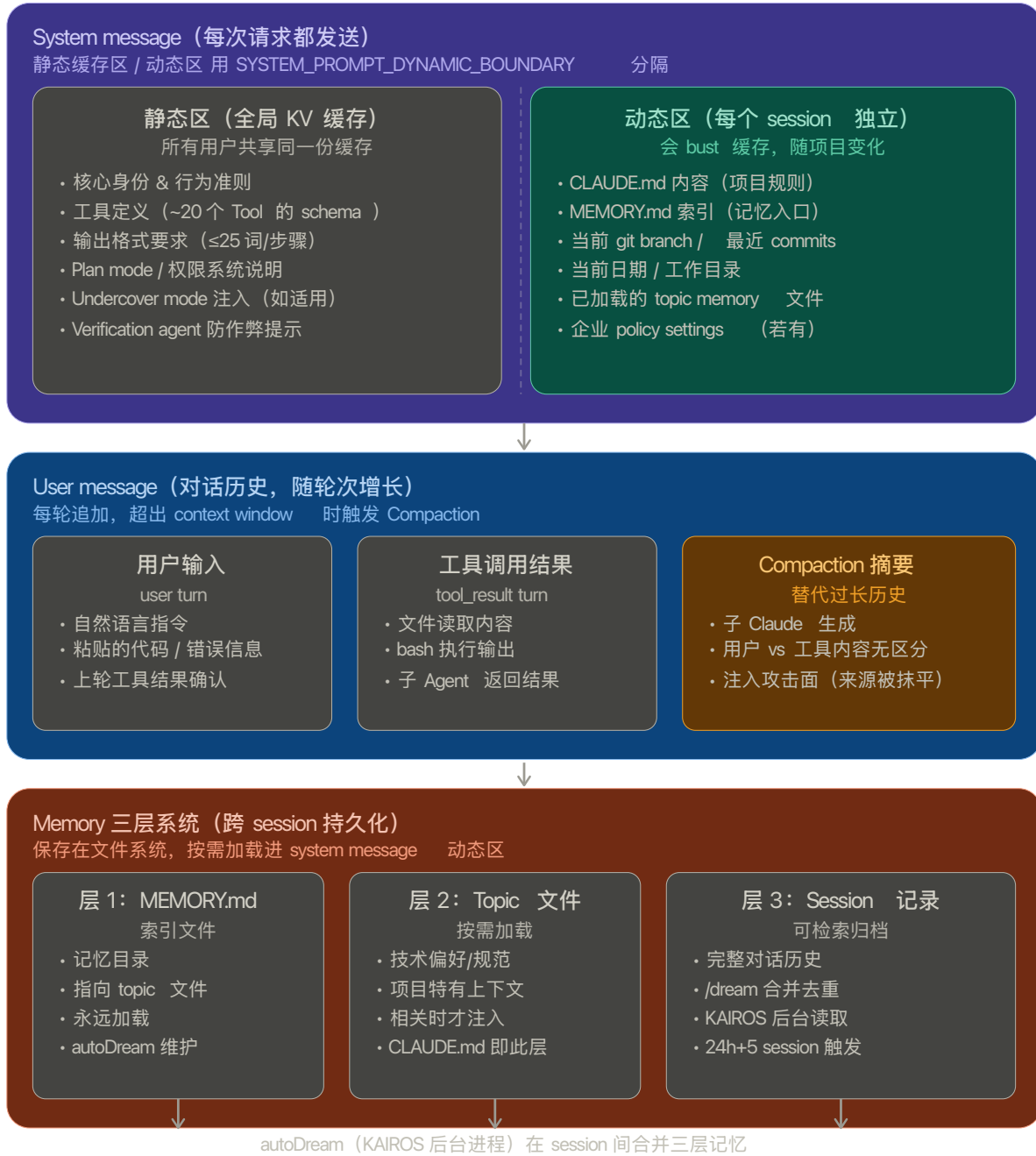


Figure 1: Claude Code 上下文架构全景

收集上下文 → 采取行动 → 验证结果

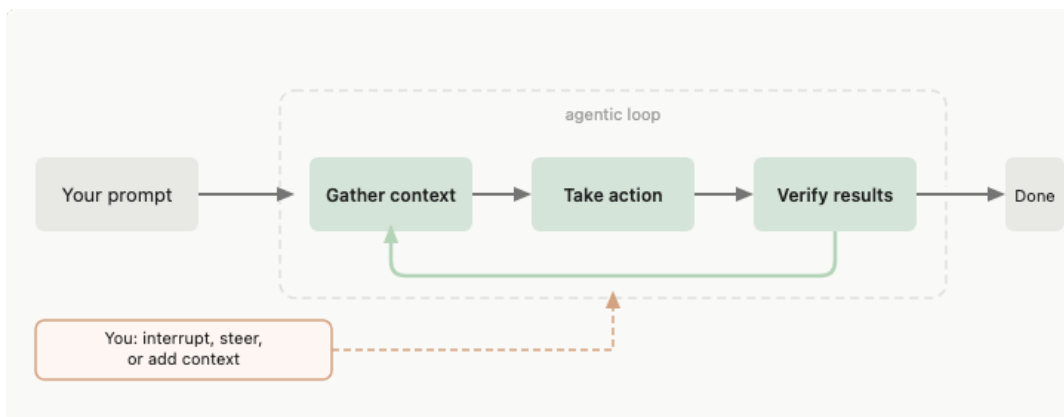


Figure 2: Agentic Loop 工作示意图

这三个阶段是交织的，而非线性的。Claude 在整个过程中持续调用工具：搜索文件来理解代码结构，调用编辑工具进行修改，运行测试来验证结果，然后根据反馈决定下一步。工具的每次调用都会返回信息，这些信息反馈回循环，指导 Claude 的下一步决策。

工具 (Tools) 与技能 (Skills) 的区别：

- Tools 是 Claude Code 的预设底层能力，规定了如何读取文件、如何写入、如何执行 Bash 命令等操作的细节。用户无法修改工具本身。
- Skills 是用户可自定义的能力封装，例如“如何使用某个特定 API”、“如何完成某类特定任务”。Skills 的运作建立在 Tools 之上。

三、如何实际使用 Claude Code

3.1 安装方式

目前有两种安装方式，两者功能完全一致：

- 官方 Native 安装：从 Anthropic 官方地址下载，适合能直接访问外网的用户。
- npm 安装：`npm install -g @anthropic-ai/claude-code`，由社区提供镜像，适合网络受限环境。

如果在安装过程中遇到依赖问题（这是最常见的坑），可以使用我整理的一键配置脚本 (zhiyu-lu.com/software) 或社区工具 `cc-switch`。

使用国内中转站接入模型时，注意潜在的安全性风险：中转站对 API 密钥和对话内容具有理论上的访问能力。

3.2 框架配置

Claude Code 的配置遵循层级制：

~/ .claude/	← 全局配置（所有项目通用）
settings.json	← 模型选择、工具权限、行为准则
skills/	← 全局 Skills 库
MEMORY.md	← Claude 自我管理的记忆索引

项目目录/ .claude/	← 项目特定配置（优先级更高）
CLAUDE.md	← 项目描述、核心限制、当前状态
settings.json	← 项目级权限覆盖

关于 **settings.json** 的几个实用建议：

建议设置 "defaultMode": "bypassPermissions"，并移除 deny 列表。否则，Claude 在执行过程中会频繁弹出确认对话框，无法真正实现长时间自主执行任务。同时，建议配置任务完成后的系统提示音——在多任务并行时，这一细节极为重要：没有提示音，你必须手动轮询每个任务的状态，无法第一时间得知某个任务已经完成。

关于 CLAUDE.md 的写法：

官方建议在以下场景更新 CLAUDE.md：Claude 犯了同一个错误第二次；代码 review 时发现 Claude 本应了解的项目约束它并不知道；你在对话框里重复输入了上个 session 也输入过的背景说明。

我自己踩过的坑是：要求越多，CLAUDE.md 就越臃肿，反而降低了有效性。我的经验是：CLAUDE.md 应该只包含最核心的项目信息（目标、核心限制、当前状态），细节性的内容让 Claude 自己去读相关文件，不要把所有东西都堆在这里。

3.3 实践中积累的几条建议

建议一：先沉淀，再写 Skill

当存在固定工作流时，先把规范写进 CLAUDE.md。如果依然频繁出现同样的错误，再考虑将规范封装成一个 Skill，供 Claude 主动调用。不要一开始就过度工程化。

建议二：Skill 保持单一职责

过长的 Skill 实际上是负效果的。例如，设计一个“完成论文全部文本撰写”的 Skill，在实践中效果甚至为负——AI 读完超长 Skill 之后，反而无法正确理解和执行。

论文各章节、各部分的细节要求差异显著，建议将这些要求拆分成一个个职责单一的小 Skill，让 Claude 在撰写过程中按需调用不同的 Skill 来完成不同部分。

建议三：先 Plan，再执行

提示词的设计已经不是 Claude Code 工作流的核心瓶颈，但没有好的上下文，AI 依然会 garbage in, garbage out。除了在 prompt 中预先提供背景，目前更推荐的做法是：在执行前让 Claude 先探索当前任务情况，输出一个完整的执行计划 (PLAN)，确认无误后再正式执行。

这一步骤可以避免大量因为对任务理解偏差而产生的无效工作。

建议四：一切皆 Prompt

Claude Code 的任何执行框架，都可以通过 prompt 来定制或替代。包括 Claude 自己的设计——什么时候 plan、如何 plan、如何执行、如何验证结果——都可以通过 CLAUDE.md 或 Skills 中的 prompt 来重新定义。这也是将 Claude Code 融入个人工作流最灵活的方式。GitHub 上有大量围绕这个主题社区资源库。

四、我的 Skill 库：以科研工作流为例

4.1 Skill 的设计原则

一个好的 Skill，本质上是给 Claude 的“任务说明书”：在什么情况下调用我，需要什么输入，期望什么输出，有哪些需要注意的细节。

Skill 文件采用 Markdown 格式，带 YAML 前言：

name: 技能名称

description: 一句话描述——用于判断这个 Skill 在什么情况下适用

使用场景

执行步骤

注意事项

Claude Code 在启动时会加载所有 Skill 的 `description` 字段。当你给它一个任务时，它会根据 `description` 判断是否需要调用某个 Skill，然后读取完整的 Skill 内容来执行。

4.2 我当前的底层 Skill 体系

经过迭代，我目前维护的底层 Skill 包括以下几类：

Skill	功能描述
<code>literature-review</code>	调用 OpenAlex API 搜索文献，筛选期刊列表和 SSRN 工作论文
<code>pyfixest-empirical</code>	使用 PyFixest 完成计量回归与实证分析 workflow
<code>academic-figure</code>	用 Matplotlib 生成符合期刊规范的图表
<code>journal-docx</code>	将 Markdown 稿件排版为指定期刊格式的 <code>.docx</code> 文件
<code>academic-tone</code>	检查并修正中文经济学论文的 AI 写作痕迹与语体规范

这些 Skill 覆盖了科研工作流中最底层、最通用的能力。我刻意没有为论文的每个章节写专门的 Skill——如建议一所说，只有当 AI 确实无法自行完成某类任务时，才值得为其写一个 Skill 来显式指导它。

4.3 一个典型工作流的展示

以“完成一节实证分析”为例，Claude Code 的实际执行路径大致如下：

1. 读取上下文：Claude 读取 `CLAUDE.md` 了解项目背景，读取数据文件的 `schema` 了解变量定义。
2. Plan：Claude 输出一个执行计划，列明将使用的回归规格、控制变量、稳健性检验方案，等待确认。
3. 执行：调用 `pyfixest-empirical` Skill，编写 Python 脚本，调用 Bash 工具执行，读取输出结果。
4. 生成图表：调用 `academic-figure` Skill，将回归结果可视化。
5. 写作：根据回归结果撰写对应文字，调用 `academic-tone` Skill 检查语体规范。
6. 排版：调用 `journal-docx` Skill，将文字与图表排版为指定格式的文件。

整个过程中，Claude 始终在同一个本地环境中操作真实的文件，读取真实的数据，生成真实的输出——这是与之前所有工具的根本区别。

结语

从提示词到工具集成，再到模型与本地环境的深度集成，这一演进过程的核心线索是：模型对执行环境的自主感知与控制能力在持续扩大。

Claude Code 并不是终点。它目前的局限同样明显：长任务中的上下文管理成本、多文件修改中的一致性保证、对非结构化数据的处理能力等，都仍需大量人工介入。

但对于有明确 workflow、可以拆解为离散步骤的科研任务，它已经能够将我从大量重复性操作中解放出来，专注于真正需要判断力的部分。这，大概就是当前 AI 工具最诚实的价值定位。